

Northeastern University
Assignment 1: EECE 7205
Professor: Dr. Fatema Nafa

Rohini Bidnurmah

NUID: 003916201

September 2026

Contents

1	Problem 1	3
1.1	Problem Statement	3
1.2	Description of Approach	3
1.2.1	Algorithm Chosen and Correctness Explanation	3
1.2.2	Pseudocode	4
1.2.3	Time and Space Complexity Derivation	4
1.3	Test Cases and Screenshots	5
2	Problem 2	6
2.1	Problem Statement	6
2.2	Description of Approach	6
2.2.1	Algorithms Chosen and Correctness Explanation	6
2.2.2	Pseudocode	7
2.2.3	Time and Space Complexity Derivation	7
2.3	Test Cases and Screenshots	8
3	Problem 3	9
3.1	Problem Statement	9
3.2	Description of Approach	9
3.2.1	Algorithm Chosen and Correctness Explanation	9
3.2.2	Pseudocode	10
3.2.3	Time and Space Complexity Derivation	10
3.3	Test Cases and Screenshots	11
4	Problem 4	12
4.1	Problem Statement	12
4.2	Description of Approach	12
4.2.1	Algorithms Chosen and Correctness Explanation	12
4.2.2	Pseudocode	13
4.2.3	Time and Space Complexity Derivation	13
4.3	Test Cases and Screenshots	14
5	Problem 5	15
5.1	Problem Statement	15
5.2	Description of Approach	15
5.2.1	Algorithm Chosen and Correctness Explanation	15
5.2.2	Pseudocode	16
5.2.3	Time and Space Complexity Derivation	16
5.3	Test Cases and Screenshots	17
6	Problem 6	18
6.1	Problem Statement	18
6.2	Description of Approach	18
6.2.1	Algorithms Chosen and Correctness Explanation	18
6.2.2	Pseudocode	19
6.2.3	Time and Space Complexity Derivation	19
6.3	Test Cases and Screenshots	20

7	Problem 7	21
7.1	Problem Statement	21
7.2	Description of Approach	21
7.2.1	Algorithm Chosen and Correctness Explanation	21
7.2.2	Pseudocode	22
7.2.3	Time and Space Complexity Derivation	22
7.3	Test Cases and Screenshots	23
8	Problem 8	24
8.1	Problem Statement	24
8.2	Description of Approach	24
8.2.1	Algorithm Chosen and Correctness Explanation	24
8.2.2	Pseudocode	25
8.2.3	Time and Space Complexity Derivation	25
8.3	Test Cases and Screenshots	26
9	Problem 9	27
9.1	Problem Statement	27
9.2	Description of Approach	27
9.2.1	Algorithm Chosen and Correctness Explanation	27
9.2.2	Pseudocode	28
9.2.3	Time and Space Complexity Derivation	28
9.3	Test Cases and Screenshots	28
10	Problem 10	29
10.1	Problem Statement	29
10.2	Approach 1: Modified Merge Sort (Divide and Conquer)	29
10.2.1	Algorithm Chosen and Correctness Explanation	29
10.2.2	Pseudocode	30
10.2.3	Time and Space Complexity Derivation	30
10.3	Approach 2: Fenwick Tree / Binary Indexed Tree (Range Query Approach)	31
10.3.1	Algorithm Chosen and Correctness Explanation	31
10.3.2	Pseudocode	32
10.3.3	Time and Space Complexity Derivation	32
10.4	Test Cases and Screenshots	33
10.5	Comparative Analysis: Merge Sort vs. Fenwick Tree	33

1 Problem 1

Description: Intervals appear in calendars, resource reservations, timelines, and scheduling systems. Overlapping intervals can be combined into a smaller set of non-overlapping intervals.

Question: Given an array of intervals `[start, end]`, sort the intervals by starting value and merge every overlapping interval. Return the resulting non-overlapping intervals.

1.1 Problem Statement

Input: A 2D vector/array `arr` of size N , where each element is an interval represented as a pair of integers `[start, end]` such that $start \leq end$.

Task: Sort the intervals by their start time in ascending order, then iterate through the sorted list and merge any adjacent overlapping or contiguous intervals where $start_i \leq end_{last}$.

Output: A 2D vector `res` containing the consolidated set of non-overlapping intervals.

Constraints: $N \geq 1$; $start, end \geq 0$; $start \leq end$ for each interval pair.

Edge Cases:

- $N = 1$: Single interval (returns itself unchanged).
- Completely overlapping intervals (e.g., $[1, 10]$ and $[2, 5] \implies [1, 10]$).
- Touching/adjacent boundaries (e.g., $[1, 3]$ and $[3, 5] \implies [1, 5]$).
- Disjoint/non-overlapping intervals (e.g., $[1, 2]$ and $[4, 5] \implies$ no merge).
- Already sorted input vs. completely reverse-sorted input.

1.2 Description of Approach

1.2.1 Algorithm Chosen and Correctness Explanation

The approach uses a two-phase strategy to merge intervals efficiently:

1. **Sorting Phase:** First, we sort the intervals based on their start values (lexicographically: sorting by $start$, using end as a tie-breaker). Sorting guarantees that for any interval i , all candidate intervals that could potentially overlap with it appear sequentially right after it. In the implemented solution, `**Merge Sort**` ($O(N \log N)$) is used to sort the intervals reliably.
2. **Linear Merging Pass:** We initialize a result vector `res` and push the first interval `arr[0]` into it. We then iterate from $i = 1$ to $N - 1$:
 - Let the current interval be `[start, end]` and the last merged interval in `res` be `[last_start, lastend]`.
 - If $start \leq lastend$, an overlap occurs. We update the endpoint of the current merged interval in `res` to be $\max(end, lastend)$.
 - Otherwise ($start > lastend$), no overlap exists, so we push `[start, end]` as a new interval into `res`.

Correctness Justification: Because intervals are sorted by start times, if $start_i > lastend$, no subsequent interval $j > i$ can overlap with the intervals in `res` before i because $start_j \geq start_i > lastend$. Thus, a single linear pass guarantees that all potential overlaps are processed and merged correctly.

1.2.2 Pseudocode

Algorithm 1 Problem 1 — Merge Overlapping Intervals

```

1: procedure MERGEINTERVALS(arr, N)
2:   MERGESORT2D(arr, 0, N - 1) ▷ Sort intervals by start value
3:   res ← empty 2D vector
4:   res.push_back(arr[0])
5:   for i ← 1 to N - 1 do
6:     start ← arr[i][0]
7:     end ← arr[i][1]
8:     lastend ← res.back()[1]
9:     if start ≤ lastend then
10:      res.back()[1] ← max(end, lastend)
11:     else
12:      res.push_back(arr[i])
13:     end if
14:   end for
15:   return res
16: end procedure

```

1.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of intervals input by the user.

1. **Input Validation Loops:** Reading and validating N interval pairs line-by-line takes $c_1 \cdot N + c_2$ operations.
2. **Merge Sort Step (mergesortfunc(arr)):** Merge Sort recursively divides the array into halves and merges them back together:
 - The recursion tree depth is $\lceil \log_2 N \rceil$.
 - At each level of the tree, the total linear merging work across all sub-arrays takes $O(N)$ operations.

Total sorting operations $T_{\text{sort}}(N) = c_3 \cdot N \log_2 N$.

3. **Result Vector Initialization:** Allocating **res** and executing **res.push_back(arr[0])** takes c_4 constant operations.
4. **Merging Pass Loop:** The loop iterates from $i = 1$ to $N - 1$ (running $N - 1$ times):
 - Variable assignments (**start**, **end**, **lastend**): 3 operations.
 - Conditional check (**start** ≤ **lastend**): 1 operation.
 - Updating **res.back()[1]** or executing **res.push_back**: 1 operation.

Total merge operations $T_{\text{merge}}(N) = c_5 \cdot (N - 1)$.

5. **Output Printing Loop:** Printing the merged intervals runs M times (where $M \leq N$ is the size of **res**), taking $c_6 \cdot M$ operations.

Total Exact Cost Function $T(N)$:

$$\begin{aligned}
 T(N) &= T_{\text{input}} + T_{\text{sort}} + T_{\text{init}} + T_{\text{merge}} + T_{\text{print}} \\
 &= (c_1 N + c_2) + c_3(N \log N) + c_4 + c_5(N - 1) + c_6 M \\
 &= c_3(N \log N) + (c_1 + c_5)N + (c_2 + c_4 - c_5) + c_6 M
 \end{aligned}$$

Dropping constant coefficients and non-dominant terms yields the final Big-O bounds:

- **Worst-Case Time Complexity** — $O(N \log N)$: Merge Sort takes $O(N \log N)$ irrespective of initial order. The subsequent linear pass takes $O(N)$. Total worst-case complexity is dominated by sorting: $T(N) \implies O(N \log N)$.
- **Best-Case Time Complexity** — $O(N \log N)$: Standard Merge Sort divides and merges unconditionally, running in $O(N \log N)$ time even on already sorted inputs.
- **Average-Case Time Complexity** — $O(N \log N)$: Merge Sort performs $O(N \log N)$ comparisons on average across random inputs.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(N)$: Merge Sort requires temporary arrays L and R during merging steps, consuming $O(N)$ auxiliary memory.

Note on Sorting Trade-offs (Heap Sort vs. Merge Sort):

- **In-place Sorting Option**: If we replace `mergesortfunc` with `heapsortfunc`, Heap Sort sorts the array in-place using $O(1)$ auxiliary space while keeping the time complexity at $O(N \log N)$.
- **Total Space Complexity with Heap Sort**: Even though Heap Sort reduces the auxiliary algorithm space to $O(1)$, storing the final merged output in `res` still requires $O(N)$ space in the worst case (when no intervals overlap). Therefore, total space complexity remains $O(N)$ overall due to the result vector, but auxiliary workspace drops to $O(1)$.
- **Output Memory** — $O(N)$: In the worst case where no intervals overlap, `res` stores all N pairs, requiring $2N$ integers.
- **Total Space Complexity**: $S(N) = S_{\text{arr}} + S_{\text{merge_aux}} + S_{\text{res}} \implies O(N)$.

1.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

Note on how the output file was created:

I created a separate `test_cases_typed_out.txt` and `test_cases_valid_only.txt` which contains written terminal inputs. The first file contains test cases testing the input validation bounds while the latter file contains just normal test cases and edge cases no input validation bounds.

A cpp file called `main_echo.cpp` was also created so I can record the input being passed into my main cpp file. This was done by using the PS terminal command:

```
g++ -std=c++17 echo_main.cpp -o main_echo.exe
Get-Content test_cases_valid_only.txt | .\main_echo.exe > results_valid.txt
```

Two result files were created for reference: `result.txt` and `results_valid.txt`. Former contains results for test cases + input validation check cases while the latter contains only test cases and edge cases. For a cleaner output verification refer to the `result_valid.txt` file.

2 Problem 2

Description: A manufacturing line labels items with three categories represented by 0, 1, and 2. The items must be grouped in category order.

Question: Given an array containing only 0, 1, and 2, sort it in place. First implement a counting-based solution. Then implement the one-pass Dutch National Flag algorithm without using `std::sort()`.

2.1 Problem Statement

Input: An integer array/vector `arr` of size N containing only elements with values 0, 1, or 2.

Task: Sort the array in-place to group all 0s, then all 1s, and finally all 2s in ascending order, without utilizing built-in sorting functions.

Output: The same input array `arr` modified in-place to be fully sorted.

Constraints: $N \geq 0$; $\forall i, arr[i] \in \{0, 1, 2\}$.

Edge Cases:

- $N = 0$: Empty array (handled safely with an immediate return).
- Array already sorted (e.g., `[0, 0, 1, 1, 2, 2]`).
- Array in reverse sorted order (e.g., `[2, 2, 1, 1, 0, 0]`).
- Array containing only a single unique element (e.g., all 0s, all 1s, or all 2s).

2.2 Description of Approach

2.2.1 Algorithms Chosen and Correctness Explanation

To solve this problem comprehensively, two distinct approaches are implemented and analyzed:

1. Approach 1: Two-Pass Counting Sort

- *Mechanism:* We initialize a frequency vector `count` of size 3 to 0. In the first pass, we iterate through `arr` and increment the frequency of each value (0, 1, or 2). In the second pass, we overwrite `arr` in-place by writing out `count[0]` zeros, `count[1]` ones, and `count[2]` twos.
- *Correctness:* Because elements are strictly limited to $\{0, 1, 2\}$, frequencies completely capture the array's distribution. Sequential overwriting guarantees a valid, in-place sort.

2. Approach 2: One-Pass Dutch National Flag (DNF) Algorithm

- *Mechanism:* Introduced by Edsger Dijkstra, this algorithm maintains four partition zones using three pointers: `low` (0 to `low - 1` holds 0s), `mid` (low to `mid - 1` holds 1s), unexamined elements (`mid` to `high`), and 2s (`high + 1` to $N - 1$). Iterating while `mid` $\leq$ `high`:
 - If `arr[mid] == 0`: Swap `arr[low]` and `arr[mid]`, then increment both `low` and `mid`.
 - If `arr[mid] == 1`: Increment `mid`.
 - If `arr[mid] == 2`: Swap `arr[mid]` and `arr[high]`, then decrement `high` (do not increment `mid`, as the swapped element must be evaluated).
- *Correctness:* Preserving pointer invariants ensures every element is placed into its correct category in a single pass.

2.2.2 Pseudocode

Algorithm 2 Problem 2 (Approach 1) — Counting Sort

```

1: procedure COUNTINGSORT012( $arr, N$ )
2:    $count \leftarrow [0, 0, 0]$  ▷ Frequency array for 0, 1, 2
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:      $count[arr[i]] \leftarrow count[arr[i]] + 1$ 
5:   end for
6:    $idx \leftarrow 0$ 
7:   for  $j \leftarrow 0$  to  $2$  do
8:     while  $count[j] > 0$  do
9:        $arr[idx] \leftarrow j$ 
10:       $idx \leftarrow idx + 1$ 
11:       $count[j] \leftarrow count[j] - 1$ 
12:    end while
13:  end for
14: end procedure

```

Algorithm 3 Problem 2 (Approach 2) — Dutch National Flag One-Pass Algorithm

```

1: procedure DUTCHNATIONALFLAG( $arr, N$ )
2:    $low \leftarrow 0$ 
3:    $mid \leftarrow 0$ 
4:    $high \leftarrow N - 1$ 
5:   while  $mid \leq high$  do
6:     if  $arr[mid] == 0$  then
7:        $swap(arr[low], arr[mid])$ 
8:        $low \leftarrow low + 1$ 
9:        $mid \leftarrow mid + 1$ 
10:    else if  $arr[mid] == 1$  then
11:       $mid \leftarrow mid + 1$ 
12:    else
13:       $swap(arr[mid], arr[high])$ 
14:       $high \leftarrow high - 1$ 
15:    end if
16:  end while
17: end procedure

```

2.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis for Approach 1 (Counting Sort): Let N be the number of elements in the array.

1. **Frequency Counting Pass:** The loop iterates N times. Each iteration performs a array index lookup and an increment, taking $c_1 \cdot N$ operations.
2. **Array Reconstruction Pass:** The outer loop runs 3 times ($j \in \{0, 1, 2\}$). The inner **while** loop executes exactly $count[j]$ times for each bucket. Since $\sum_{j=0}^2 count[j] = N$, the inner loop assignments run N times total, taking $c_2 \cdot N + c_3$ operations.

Exact Cost Function for Approach 1:

$$T_{\text{count}}(N) = c_1N + c_2N + c_3 = (c_1 + c_2)N + c_3 \implies O(N)$$

Step-by-Step Time Cost Analysis for Approach 2 (Dutch National Flag): Let N be the number of elements in the array.

1. **Single-Pass Iteration Loop:** The `while` condition checks if $\text{mid} \leq \text{high}$. Initially, $\text{high} - \text{mid} = N - 1$.
2. In every iteration of the loop, either `mid` increases by 1 or `high` decreases by 1.
3. Consequently, the quantity $(\text{high} - \text{mid})$ strictly decreases by at least 1 per iteration.
4. The loop executes at most N times, performing constant c_4 operations (comparisons and swaps) per iteration.

Exact Cost Function for Approach 2:

$$T_{\text{DNF}}(N) \leq c_4 N \implies O(N)$$

Summary of Big-O Time Complexities:

- **Approach 1 (Counting Sort):**
 - Worst-Case: $O(N)$
 - Best-Case: $O(N)$
 - Average-Case: $O(N)$
- **Approach 2 (Dutch National Flag):**
 - Worst-Case: $O(N)$
 - Best-Case: $O(N)$
 - Average-Case: $O(N)$

Space Complexity Derivation for Both Approaches:

- **Approach 1 (Counting Sort) Space Complexity:** Allocates a fixed-size array `count` of 3 integers to store element frequencies. Since the memory for `count` does not grow with N , auxiliary space is $O(1)$. Total memory including input storage is $O(N)$.
- **Approach 2 (Dutch National Flag) Space Complexity:** Operates strictly in-place, relying solely on three scalar pointer variables (`low`, `mid`, `high`). Auxiliary memory consumption is $O(1)$. Total memory including input storage is $O(N)$.

Both approaches achieve $O(1)$ auxiliary space. However, DNF requires only a single pass over the array (with in-place swaps), whereas the counting approach requires two passes (one to count, one to overwrite), giving DNF a smaller constant factor in practice.

2.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

3 Problem 3

Description: A custom ordering is sometimes required when the normal numeric order does not produce the desired combined result.

Question: Given a vector of nonnegative integers, arrange them so that their concatenation forms the largest possible number. Return the answer as a string.

3.1 Problem Statement

Input: A vector `arr` of N non-negative integers.

Task: Custom-sort the array using the string concatenation criterion ($A + B > B + A$) such that concatenating the integers sequentially produces the lexicographically largest numeric string.

Output: A string representing the largest possible concatenated number.

Constraints: $N \geq 0$; $\forall i, arr[i] \geq 0$.

Edge Cases:

- $N = 0$: Empty vector (returns empty string "").
- All elements are zero (e.g., $[0, 0, 0] \implies "0"$, preventing outputs like "000").
- Numbers with shared prefixes of varying lengths (e.g., 3 and 30 $\implies "330"$ because "330" $>$ "303").
- Single element input (e.g., $[9] \implies "9"$).

3.2 Description of Approach

3.2.1 Algorithm Chosen and Correctness Explanation

Standard numeric or lexicographical comparison fails for this problem (for example, numerically $30 > 3$, but concatenating them as "330" produces a larger value than "303").

1. **Custom Comparator Formulation:** For any two non-negative integers a and b , let $\text{str}(a)$ and $\text{str}(b)$ be their string representations. a should precede b in the sorted order if and only if:

$$\text{str}(a) + \text{str}(b) > \text{str}(b) + \text{str}(a)$$

2. **Heap Sort Integration:** In-place **Heap Sort** using this custom predicate orders the vector such that elements forming larger concatenated prefixes are placed first.
3. **Leading Zero Handling:** After sorting, if the largest element `arr[0]` is 0, all subsequent elements must also be 0. The algorithm immediately returns "0" to prevent generating redundant leading zeros.
4. **Concatenation:** Otherwise, the algorithm iterates through the sorted array and prints/concatenates each element sequentially into the final string output.

Correctness Justification: The custom concatenation comparator defines a total ordering (transitive and anti-symmetric). Sorting the elements according to this relation guarantees that any adjacent inversion would decrease or keep equal the total concatenated string value, proving global optimality.

3.2.2 Pseudocode

Algorithm 4 Problem 3 — Largest Number via Heap Sort with Custom Comparator

```

1: procedure LARGESTNUMBER(arr, N)
2:   if N == 0 then
3:     return ""
4:   end if
5:   HEAPSORTCMP(arr, Comparator)           ▷ Comparator checks: str(a) + str(b) > str(b) + str(a)
6:   if arr[0] == 0 then
7:     return "0"
8:   end if
9:   result ← empty string
10:  for i ← 0 to N − 1 do
11:    result ← result + to_string(arr[i])
12:  end for
13:  return result
14: end procedure

```

3.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of integers and K be the maximum number of digits in any integer (where K is bounded and small, $K = O(1)$).

1. **Heap Sort Phase:** Building the heap and performing N extractions takes $O(N \log N)$ comparison operations.
2. **Comparison Operator Cost:** Each comparison converts two integers to strings and concatenates them. String conversion and string comparison take $O(K)$ time.

Total sorting time cost:

$$T_{\text{sort}}(N) = c_1 \cdot (N \log N) \cdot K = O(N \log N) \quad (\text{since } K \text{ is constant})$$

3. **Output Concatenation Pass:** Iterating through the sorted array of length N to construct the output string takes $c_2 \cdot N$ string append operations.

Total Exact Cost Function $T(N)$:

$$T(N) = T_{\text{sort}}(N) + T_{\text{concat}}(N) = c_1(N \log N) + c_2N \implies O(N \log N)$$

Summary of Big-O Time Complexities:

- **Worst-Case Time Complexity** — $O(N \log N)$: Heap Sort maintains guaranteed $O(N \log N)$ run time regardless of initial array ordering.
- **Best-Case Time Complexity** — $O(N \log N)$: Heapification and iterative extraction run in $O(N \log N)$ steps even if the input is already in optimal order.
- **Average-Case Time Complexity** — $O(N \log N)$: Average case is dominated by the $O(N \log N)$ sorting operations.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(1)$ (or $O(K)$ **temporary string allocations**): Because Heap Sort operates strictly in-place on the vector `arr`, no additional arrays are allocated for sorting. Temporary string allocations during comparator evaluations take constant $O(K) = O(1)$ space.

- **Output Memory** — $O(N)$: The final output string stores N numbers concatenated together, requiring $O(N \cdot K) = O(N)$ memory.
- **Total Space Complexity**: $S(N) = O(1)$ auxiliary workspace, $O(N)$ total memory including the result string.

3.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

4 Problem 4

Description: Anagrams contain the same characters with the same frequencies but in a different order. Grouping them is useful in text indexing and search.

Question: Given a vector of strings, group the anagrams together. The order of groups does not matter, but words within each group should retain their original input order.

4.1 Problem Statement

Input: A vector of strings `strs` of size N , containing lowercase words from $[a - z]$.

Task: Group strings that are anagrams of one another into lists, preserving the relative insertion order of strings within each group.

Output: A list of vectors/groups, where each group contains all words that are mutual anagrams.

Constraints: $N \geq 0$; each string length $K \geq 0$; characters are strictly lowercase English letters (a to z).

Edge Cases:

- $N = 0$: Empty input vector (returns empty output list `[]`).
- $K = 0$: Empty string inputs (e.g., `["", ""]` $\implies$ grouped together).
- Single word inputs (e.g., `["a"]` $\implies$ `[["a"]]`).
- Words with identical character sets but different frequencies (e.g., `"aab"` vs. `"abb"` $\implies$ distinct groups).
- Input with no anagram pairs (all words belong to their own unique group).

4.2 Description of Approach

4.2.1 Algorithms Chosen and Correctness Explanation

Two main approaches exist for generating the canonical key used to group anagrams in a hash map:

1. Approach 1: String Sorting Key ($O(N \cdot K \log K)$)

- *Mechanism:* For each string, sort its characters lexicographically to form a key (e.g., `"eat"` $\implies$ `"aet"`). Use an `unordered_map<string, vector<string>>` to append each string to the bucket corresponding to its sorted key.
- *Drawback:* Sorting each string of length K adds $O(K \log K)$ operations per word.

2. Approach 2: Character Frequency Key / Frequency Counting ($O(N \cdot K)$)

- *Mechanism:* Since inputs are restricted to lowercase English letters ($a - z$), an integer array of size 26 tracks character frequencies for each string. We convert this frequency array into a unique formatted string key (e.g., `"#1#0#0...#1"`). We then insert the original string into an `unordered_map` indexed by this frequency key.
- *Correctness:* Anagrams are defined as strings having identical character counts for every character. Formatting the 26 character counts into a delimited string guarantees a unique string key for each equivalence class. Pushing words into the map sequentially preserves original relative input ordering within each bucket.

4.2.2 Pseudocode

Algorithm 5 Problem 4 (Approach 1) — Group Anagrams via Sorted Key

```

1: procedure GROUPANAGRAMSORT(strs, N)
2:   groups  $\leftarrow$  empty hash map mapping string to vector of strings
3:   for i  $\leftarrow$  0 to N - 1 do
4:     key  $\leftarrow$  strs[i]
5:     SORT(key) ▷ Sort characters in  $O(K \log K)$ 
6:     groups[key].push_back(strs[i])
7:   end for
8:   return groups
9: end procedure

```

Algorithm 6 Problem 4 (Approach 2) — Group Anagrams via Frequency Key

```

1: procedure GROUPANAGRAMSFREQ(strs, N)
2:   groups  $\leftarrow$  empty hash map mapping string to vector of strings
3:   for i  $\leftarrow$  0 to N - 1 do
4:     count  $\leftarrow$  array of size 26 initialized to 0
5:     for each c in strs[i] do
6:       count[c - 'a']  $\leftarrow$  count[c - 'a'] + 1
7:     end for
8:     key  $\leftarrow$  empty string
9:     for k  $\leftarrow$  0 to 25 do
10:      key  $\leftarrow$  key + "#" + to_string(count[k])
11:    end for
12:    groups[key].push_back(strs[i])
13:  end for
14:  return groups
15: end procedure

```

4.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis for Approach 1 (Sorted Key): Let N be the number of strings and K be the maximum length of a string.

1. Iterating over N strings: N steps.
2. Sorting each string of length K : $c_1 \cdot K \log K$ operations.
3. Hash map lookup and insertion for key of length K : $c_2 \cdot K$ operations.

$$T_{\text{sort_key}}(N, K) = N \cdot (c_1 K \log K + c_2 K) \implies O(N \cdot K \log K)$$

Step-by-Step Time Cost Analysis for Approach 2 (Frequency Key): Let N be the number of strings and K be the maximum length of a string.

1. **Frequency Counting Pass:** For each string, counting K characters takes $c_3 \cdot K$ operations.
2. **Key Construction Pass:** Iterating through the fixed 26-element array to construct the formatted string key takes $26 \cdot c_4 = O(1)$ operations.
3. **Hash Map Insertion:** Inserting the string into the map takes $c_5 \cdot K$ average time for hashing and vector push.

Exact Cost Function for Approach 2:

$$T_{\text{freq_key}}(N, K) = N \cdot (c_3 K + 26c_4 + c_5 K) = (c_3 + c_5)NK + 26c_4 N \implies O(N \cdot K)$$

Summary of Big-O Time Complexities:

- **Approach 1 (Sorting-based):** Worst, Best, and Average-case: $O(N \cdot K \log K)$.
- **Approach 2 (Frequency-based):** Worst, Best, and Average-case: $O(N \cdot K)$ (optimal linear time relative to total character count).

Space Complexity Derivation for Both Approaches:

- **Approach 1 (Sorted Key):**
 - Auxiliary Key Space: Temporary strings of length K require $O(K)$ space.
 - Hash Map Storage: Stores all N strings of length K in buckets, requiring $O(N \cdot K)$ space.
 - Total Space Complexity: $S(N, K) = O(N \cdot K)$.
- **Approach 2 (Frequency Key):**
 - Auxiliary Frequency Space: A fixed 26-element integer array per string takes $O(1)$ space.
 - Key Length Space: The formatted key string has maximum length bounded by $26 \times \text{digits} \approx 80$ chars = $O(1)$ space.
 - Hash Map Storage: Storing all strings in the map requires $O(N \cdot K)$ memory.
 - Total Space Complexity: $S(N, K) = O(N \cdot K)$.

4.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

5 Problem 5

Description: Calendar software must determine whether a person or resource has been assigned to overlapping meetings.

Question: Given meeting intervals `[start, end]`, determine whether one person can attend every meeting. Meetings that begin exactly when another ends are allowed.

5.1 Problem Statement

Input: A 2D vector/array `arr` of size N , where each element is an interval represented as a pair of integers `[start, end]` with $start \leq end$.

Task: Determine if any two meeting intervals overlap. Return `true` if all meetings can be attended without scheduling conflicts, and `false` otherwise. Adjacent meetings where $start_i = end_{i-1}$ are permitted.

Output: A boolean decision printed as a status message indicating whether the person can attend all meetings (`true`) or if a schedule overlap exists (`false`).

Constraints: $N \geq 0$; $start, end \geq 0$; $start \leq end$ for each meeting interval.

Edge Cases:

- $N = 0$ or $N = 1$: Zero or single meeting (always returns `true` as no conflicts can exist).
- Back-to-back adjacent meetings (e.g., `[0, 5]` and `[5, 10]` $\implies$ `true`, allowed per problem statement).
- Strictly overlapping meetings (e.g., `[0, 30]` and `[5, 10]` $\implies$ `false`).
- Identical zero-duration meeting intervals (e.g., `[4, 4]` and `[4, 4]` $\implies$ `false`, scheduling conflict at time 4).
- Meetings given in unsorted or reverse-sorted order.

5.2 Description of Approach

5.2.1 Algorithm Chosen and Correctness Explanation

To detect overlaps efficiently without performing an $O(N^2)$ pairwise comparison:

1. **In-Place Sorting Phase:** We first sort the meeting intervals in-place using **Heap Sort** (`heapsortfunc(arr)`) based on start times (lexicographically sorting by `start`, with `end` as tie-breaker). Sorting arranges meetings chronologically so that any potential scheduling conflict occurs between consecutive intervals.
2. **In-Place Conflict Check Pass:** We maintain a boolean flag `can_attend` initialized to `true`. We iterate linearly from $i = 1$ to $N - 1$, comparing the current meeting `arr[i]` with the preceding meeting `arr[i-1]`:
 - If `arr[i][0] < arr[i-1][1]`, the current meeting starts strictly before the previous meeting finishes, indicating an overlap.
 - Alternatively, if both meetings are identical (e.g., duplicate intervals `[4, 4]` and `[4, 4]`), a conflict is identified.
 - When an overlap is detected, `can_attend` is set to `false` and the loop terminates early.

Correctness Justification and Memory Efficiency: Sorting guarantees that for all $j > i$, $start_j \geq start_i$. Thus, if $start_i \geq end_{i-1}$ holds for all adjacent pairs i , then no meeting $j > i$ can start before end_{i-1} , proving that no overlaps exist anywhere in the schedule. Furthermore, using Heap Sort and a simple boolean flag avoids allocating a result vector, optimizing space to strictly $O(1)$ auxiliary memory.

5.2.2 Pseudocode

Algorithm 7 Problem 5 — Can Attend All Meetings

```

1: procedure CANATTENDMEETINGS(arr, N)
2:   if  $N \leq 1$  then
3:     return true
4:   end if
5:   HEAPSORT2D(arr, N)                                ▷ In-place Heap Sort takes  $O(N \log N)$  time,  $O(1)$  space
6:   can_attend  $\leftarrow$  true
7:   for  $i \leftarrow 1$  to  $N - 1$  do
8:     if  $arr[i][0] < arr[i-1][1]$  or  $(arr[i][0] == arr[i-1][0] \text{ and } arr[i][1] == arr[i-1][1])$  then
9:       can_attend  $\leftarrow$  false
10:      break
11:    end if
12:  end for
13:  return can_attend
14: end procedure

```

5.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the total number of meeting intervals.

- Input Validation and Reading:** Reading N interval pairs line-by-line takes $c_1 \cdot N + c_2$ operations.
- Heap Sort Step (heapsortfunc(arr)):**
 - Heapification phase constructs a max-heap in $O(N)$ operations.
 - Extraction phase performs $N - 1$ pop operations, each taking $O(\log N)$ time to sift down elements.

Total sorting time cost $T_{\text{sort}}(N) = c_3 \cdot N \log_2 N$.

- Linear Conflict Checking Pass:** Iterates at most $N - 1$ times across adjacent elements:
 - Array element access and boundary comparison: c_4 operations.
 - Early exit on first detected overlap: $O(1)$ minimum, $O(N)$ maximum steps.

Total linear scan operations $T_{\text{scan}}(N) \leq c_4 \cdot (N - 1)$.

Total Exact Cost Function $T(N)$:

$$T(N) = T_{\text{input}} + T_{\text{sort}} + T_{\text{scan}} = (c_1 N + c_2) + c_3(N \log N) + c_4(N - 1) \implies O(N \log N)$$

Summary of Big-O Time Complexities:

- Worst-Case Time Complexity** — $O(N \log N)$: Occurs when all meetings are non-overlapping and the loop inspects all $N - 1$ pairs after Heap Sort finishes in $O(N \log N)$ time.
- Best-Case Time Complexity** — $O(N \log N)$: Even if a conflict occurs on the first comparison ($i = 1$), the preliminary Heap Sort step unconditionally takes $O(N \log N)$ comparisons.
- Average-Case Time Complexity** — $O(N \log N)$: Dominated by the $O(N \log N)$ sorting step across random meeting inputs.

Space Complexity Derivation:

- Auxiliary Memory** — $O(1)$: Heap Sort operates strictly in-place directly on **arr**, requiring zero temporary arrays. The conflict checking phase uses a single primitive boolean flag (**can_attend**), keeping auxiliary space strictly at $O(1)$.
- Total Space Complexity:** $S(N) = S_{\text{arr}} + S_{\text{aux}} = 2N + O(1) \implies O(N)$ total space including input storage.

5.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

6 Problem 6

Description: Navigation, robotics, and spatial systems often need to identify the nearest locations or sensor readings.

Question: Given points (x, y) and an integer k , return the k points closest to the origin. Compare a sorting solution with a heap-based solution.

6.1 Problem Statement

Input: A 2D vector/array `arr` of N spatial points where each point is $[x, y]$, and an integer k representing the number of nearest points to retrieve.

Task: Calculate the squared Euclidean distance $d^2 = x^2 + y^2$ from the origin $(0, 0)$ for each point, and select the k points with the smallest distances.

Output: A list of k points closest to the origin.

Constraints: $N \geq 0$; $1 \leq k \leq N$; $x, y \in \mathbb{Z}$.

Edge Cases:

- $N = 0$: Empty point array (handled safely).
- $k = N$: All points must be returned; no heap operations needed.
- Points with duplicate distances (e.g., $(1, 2)$ and $(2, 1)$ both have $d^2 = 5$; any valid k elements are accepted).
- Points located directly at the origin $(0, 0)$ ($d^2 = 0$).
- Points located in negative quadrants (e.g., $(-3, -4) \implies d^2 = 25$).

6.2 Description of Approach

6.2.1 Algorithms Chosen and Correctness Explanation

To solve this problem, we evaluate two approaches:

1. Approach 1: Full Array Sorting ($O(N \log N)$)

- *Mechanism:* Compute $d^2 = x^2 + y^2$ for all N points. Sort all N elements by d^2 in ascending order using an $O(N \log N)$ sorting algorithm, then slice the first k elements.
- *Drawback:* Performs excessive work by fully ordering all $N - k$ distant points, which are discarded.

2. Approach 2: Max-Heap of Size k ($O(N \log k)$)

- *Mechanism:* Maintain a max-heap (`std::priority_queue`) storing pairs of $(\text{distance}^2, \text{index})$. As we iterate through each of the N points, we compute its squared distance and push it onto the heap. Whenever the heap size exceeds k , we pop the top element (which is the point with the largest distance currently in the heap).
- *Correctness:* At any step, the max-heap retains exactly the k smallest distances seen so far. Popping the maximum element whenever size $> k$ ensures that after inspecting all N points, the heap holds the overall k closest points to the origin.

6.2.2 Pseudocode

Algorithm 8 Problem 6 (Approach 1) — k Closest Points via Full Sorting

```

1: procedure KCLOSESTSORT( $arr, N, k$ )
2:   if  $k == N$  then
3:     return  $arr$ 
4:   end if
5:   SORT( $arr$ , Comparator:  $x_1^2 + y_1^2 < x_2^2 + y_2^2$ )
6:    $res \leftarrow$  first  $k$  elements of  $arr$ 
7:   return  $res$ 
8: end procedure

```

Algorithm 9 Problem 6 (Approach 2) — k Closest Points via Max-Heap

```

1: procedure KCLOSESTHEAP( $arr, N, k$ )
2:   if  $k == N$  then
3:     return  $arr$ 
4:   end if
5:    $maxheap \leftarrow$  empty Max Priority Queue storing ( $dist^2$ , index)
6:   for  $i \leftarrow 0$  to  $N - 1$  do
7:      $dist \leftarrow arr[i][0]^2 + arr[i][1]^2$ 
8:      $maxheap.push(\langle dist, i \rangle)$ 
9:     if  $maxheap.size() > k$  then
10:       $maxheap.pop()$  ▷ Evicts the largest distance among the top  $k + 1$ 
11:    end if
12:  end for
13:   $res \leftarrow$  empty vector
14:  while  $maxheap$  is not empty do
15:     $idx \leftarrow maxheap.top().second$ 
16:     $maxheap.pop()$ 
17:     $res.push\_back(arr[idx])$ 
18:  end while
19:  return  $res$ 
20: end procedure

```

6.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis for Approach 1 (Full Sorting): Let N be the total number of points.

1. Computing distances for all N points: $c_1 \cdot N$ operations.
2. Sorting the array of size N : $c_2 \cdot N \log_2 N$ operations.
3. Slicing the top k elements: $c_3 \cdot k$ operations.

$$T_{\text{sort}}(N, k) = c_1 N + c_2 (N \log N) + c_3 k \implies O(N \log N)$$

Step-by-Step Time Cost Analysis for Approach 2 (Max-Heap): Let N be the total number of points and k be the number of closest points to extract.

1. **Special Case ($k = N$):** Directly prints all elements without heap operations in $c_4 \cdot N \implies O(N)$ time.
2. **General Case ($k < N$):**

- The loop executes N times. In each iteration, calculating distance takes $O(1)$ arithmetic operations.
- Inserting into a heap of max size $k + 1$ takes $O(\log k)$ time.
- Popping from the heap when size reaches $k + 1$ takes $O(\log k)$ time.

Total insertion/eviction loop cost: $T_{\text{loop}}(N, k) = c_5 \cdot N \log_2 k$.

3. **Extraction Phase:** Popping k elements from the heap takes $c_6 \cdot k \log_2 k$ operations.

Exact Cost Function for Approach 2:

$$T_{\text{heap}}(N, k) = c_5(N \log k) + c_6(k \log k) \implies O(N \log k)$$

Summary of Big-O Time Complexities:

- **Approach 1 (Sorting):** $O(N \log N)$ across worst, best, and average cases.
- **Approach 2 (Max-Heap):** $O(N \log k)$ across worst, best, and average cases. When $k \ll N$, $O(N \log k)$ is significantly faster than $O(N \log N)$.

Space Complexity Derivation for Both Approaches:

- **Approach 1 (Sorting):**
 - Auxiliary Memory: Sorting in-place takes $O(1)$ extra space (or $O(N)$ if using Merge Sort). Output array takes $O(k)$ space.
 - Total Space Complexity: $S(N, k) = O(k)$ auxiliary workspace, $O(N)$ total space including input.
- **Approach 2 (Max-Heap):**
 - Auxiliary Memory: The max-heap retains at most $k + 1$ elements at any given time, taking strictly $O(k)$ auxiliary space.
 - Total Space Complexity: $S(N, k) = O(k)$ auxiliary workspace, $O(N)$ total space including input.

6.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

7 Problem 7

Description: Monitoring and analytics systems often rank events, error codes, or queries by frequency.

Question: Given an integer array and an integer k , return the k most frequent values. If frequencies tie, return the smaller value first.

7.1 Problem Statement

Input: An integer array `arr` of size N , and an integer k representing the number of top frequent elements to return.

Task: Count the frequency of each unique element, then identify the k most frequent elements. Tie-breaking rule: if two elements have equal frequencies, prefer the smaller numeric value.

Output: A list of k pairs/elements ordered from most frequent to least frequent (respecting tie-breaker rules).

Constraints: $N \geq 0$; $1 \leq k \leq U$ (where $U \leq N$ is the number of unique elements in `arr`).

Edge Cases:

- $N = 0$: Empty array (handled safely).
- $k = U$: All unique elements requested.
- Frequency ties (e.g., elements 5 and 3 both appear 2 times $\implies$ 3 takes precedence over 5).
- Array with all identical elements (e.g., $[2, 2, 2] \implies U = 1, k = 1$).
- Array where all elements are unique (all frequencies equal 1).

7.2 Description of Approach

7.2.1 Algorithm Chosen and Correctness Explanation

Similar to Problem 6, this problem can be solved either by full sorting or using a min-heap:

- **Alternative Sorting Approach ($O(U \log U)$):** Construct a frequency map using an `unordered_map`, transfer all U unique element pairs (frequency, value) into a vector, and sort the entire vector. While straightforward, sorting all U elements wastes time ordering low-frequency items that are discarded.
- **Chosen Min-Heap Approach ($O(N + U \log k)$):**
 1. **Frequency Map Pass:** Iterate over the input array of size N to build a hash map storing {value $\rightarrow$ frequency}.
 2. **Custom Bounded Min-Heap:** Maintain a min-heap storing pairs of (frequency, value) capped at size k . The custom comparator dictates heap priorities:
 - Elements with lower frequencies stay at the top (so they are evicted first when heap size exceeds k).
 - On frequency ties ($\text{freq}_A == \text{freq}_B$), the element with the larger value is placed at the top so that larger numbers are evicted first, preserving smaller values per the tie-breaker rule.
 3. **Extraction & Reversal:** Extract elements from the min-heap into a vector and reverse it using a two-pointer approach (`left`, `right`) to present the output in descending order of frequency.

Why Min-Heap is Superior: The bounded min-heap solution is significantly better than full sorting. By processing frequencies into a heap capped at size k , it reduces the sorting complexity from $O(U \log U)$ down to $O(U \log k)$. In terms of space, keeping only k elements in the heap consumes $O(k)$ auxiliary memory rather than allocating memory for all U unique elements in a vector.

7.2.2 Pseudocode

Algorithm 10 Problem 7 — Top k Frequent Elements via Bounded Min-Heap

```

1: procedure TOPKFREQUENT( $arr, N, k$ )
2:    $freq \leftarrow$  empty hash map mapping integer to integer
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:      $freq[arr[i]] \leftarrow freq[arr[i]] + 1$ 
5:   end for
6:    $minheap \leftarrow$  empty Min-Heap capped at size  $k$  with Custom Comparator
7:   for each pair  $\langle val, count \rangle$  in  $freq$  do
8:      $minheap.push(\langle count, val \rangle)$ 
9:     if  $minheap.size() > k$  then
10:       $minheap.pop()$  ▷ Evicts smallest freq, or larger value on tie
11:     end if
12:   end for
13:    $res \leftarrow$  empty vector
14:   while  $minheap$  is not empty do
15:      $res.push\_back(minheap.top())$ 
16:      $minheap.pop()$ 
17:   end while
18:   REVERSE( $res$ ) ▷ Reverse to get descending frequency order
19:   return  $res$ 
20: end procedure

```

7.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the total number of elements in the array and U be the number of unique elements ($U \leq N$).

1. **Frequency Counting Pass:** Iterating through N elements to populate the hash map takes $c_1 \cdot N$ operations.

2. **Heap Insertion/Eviction Phase:**

- Iterates U times across all unique key-value pairs.
- Each heap push and potential eviction pop operates on a heap of max size $k + 1$, taking $O(\log k)$ time per unique element.

Total heap processing cost: $T_{\text{heap}}(U, k) = c_2 \cdot U \log_2 k$.

3. **Extraction and Reversal Phase:**

- Extracting k items takes $c_3 \cdot k \log_2 k$ operations.
- Two-pointer array reversal takes $c_4 \cdot k$ operations.

Total Exact Cost Function $T(N, U, k)$:

$$T(N, U, k) = c_1 N + c_2 (U \log k) + c_3 (k \log k) + c_4 k \implies O(N + U \log k)$$

Summary of Big-O Time Complexities:

- **Sorting Approach:** $O(N + U \log U)$ worst, best, and average-case.
- **Bounded Min-Heap Approach (Chosen):** $O(N + U \log k)$ worst, best, and average-case. When $k \ll U$, $O(N + U \log k)$ provides significant speedups.

Space Complexity Derivation:

- **Frequency Map Storage:** Storing U unique elements in `unordered_map` requires $O(U)$ memory.
- **Auxiliary Heap Memory:** The min-heap retains at most $k + 1$ elements at any given time, taking strictly $O(k)$ extra space.
- **Output Vector Memory:** Stores the k result pairs, taking $O(k)$ memory.
- **Total Space Complexity:** $S(N, U, k) = O(U + k) = O(U)$ auxiliary workspace, $O(N)$ total space including input.

7.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

8 Problem 8

Description: Distributed systems may produce multiple individually sorted streams that must be combined into one ordered result.

Question: Given k sorted integer arrays, merge them into one sorted array. Design a solution that does not repeatedly sort the complete output.

8.1 Problem Statement

Input: A collection of k sorted integer arrays $arr_0, arr_1, \dots, arr_{k-1}$ with total elements $N = \sum_{i=0}^{k-1} |arr_i|$.

Task: Merge all k arrays into a single, fully sorted integer array without repeatedly running a complete sort over all N elements.

Output: A single sorted array `merged` of length N containing all elements from all k input arrays in non-decreasing order.

Constraints: $k \geq 1$; each sub-array arr_i is individually sorted in non-decreasing order; $N \geq 0$.

Edge Cases:

- $k = 1$: Single array input (returns a copy/reference immediately).
- Some or all sub-arrays are empty (e.g., $arr_i = []$).
- Sub-arrays of varying/unequal sizes (e.g., arr_0 has size 100, arr_1 has size 1).
- Disjoint value ranges across sub-arrays (e.g., $arr_0 = [1, 2]$, $arr_1 = [10, 20]$).
- Duplicate elements across multiple sub-arrays.

8.2 Description of Approach

8.2.1 Algorithm Chosen and Correctness Explanation

A naive approach would append all N elements into one array and run Merge Sort or Heap Sort, taking $O(N \log N)$ time. This ignores the fact that each of the k sub-arrays is already sorted.

1. **Min-Heap Multiway Merge ($O(N \log k)$):** We leverage a Min-Heap (`priority_queue`) storing custom node structures (`heapnode`) containing three fields:

- `val`: The integer value itself.
- `idx`: The index of the sub-array from which the value originated ($0 \leq \text{idx} < k$).
- `elemidx`: The current index of the element within that specific sub-array.

2. **Merging Mechanics:**

- **Initialization:** Insert the first element ($arr[i][0]$) of every non-empty sub-array into the Min-Heap. The heap size is at most k .
- **Iterative Extraction:** Pop the top node `cur` (which contains the minimum element currently available across all active sub-arrays) and push `cur.val` into `merged`.
- **Stream Advance:** If the sub-array `cur.idx` has remaining elements ($\text{cur.elemidx} + 1 < \text{arr}[\text{cur.idx}].\text{size}()$), insert its next element into the Min-Heap.
- Repeat until the Min-Heap is empty.

Correctness Justification: Because each sub-array is pre-sorted, the overall minimum element among all unmerged values must always be among the head elements of the k sub-arrays currently tracked in the Min-Heap. Extracting the minimum and advancing only that specific sub-array's pointer maintains this invariant throughout all N steps.

8.2.2 Pseudocode

Algorithm 11 Problem 8 — Merge k Sorted Arrays via Min-Heap

```

    HeapNode
1:  $val \leftarrow$  integer value
2:  $idx \leftarrow$  sub-array index
3:  $elemidx \leftarrow$  element index within sub-array
4:
5: procedure MERGEKSORTEDARRAYS( $arr, k$ )
6:    $minheap \leftarrow$  Min-Heap of HeapNodes sorted by  $val$ 
7:    $total \leftarrow 0$ 
8:   for  $i \leftarrow 0$  to  $k - 1$  do
9:     if  $arr[i]$  is not empty then
10:       $minheap.push(HeapNode(arr[i][0], i, 0))$ 
11:       $total \leftarrow total + |arr[i]|$ 
12:    end if
13:  end for
14:   $merged \leftarrow$  empty vector reserved to size  $total$ 
15:  while  $minheap$  is not empty do
16:     $cur \leftarrow minheap.top()$ 
17:     $minheap.pop()$ 
18:     $merged.push\_back(cur.val)$ 
19:     $nextIdx \leftarrow cur.elemidx + 1$ 
20:    if  $nextIdx < |arr[cur.idx]|$  then
21:       $minheap.push(HeapNode(arr[cur.idx][nextIdx], cur.idx, nextIdx))$ 
22:    end if
23:  end while
24:  return  $merged$ 
25: end procedure

```

8.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let k be the number of sorted arrays and N be the total number of elements across all k arrays ($N = \sum_{i=0}^{k-1} |arr_i|$).

1. **Min-Heap Initialization:** Pushing the first element of up to k sub-arrays into the heap takes $c_1 \cdot k \log_2 k$ operations.

2. **Extraction and Refill Loop:**

- The **while** loop executes exactly N times (once per total element).
- In each iteration, popping the minimum node from a heap of maximum size k takes $c_2 \cdot \log_2 k$ operations.
- Pushing the replacement element from the same sub-array takes $c_3 \cdot \log_2 k$ operations.
- Appending to **merged** (pre-reserved) takes $c_4 = O(1)$ constant time.

Total loop time cost: $T_{\text{loop}}(N, k) = N \cdot (c_2 \log k + c_3 \log k + c_4) = c_5 \cdot N \log_2 k$.

Total Exact Cost Function $T(N, k)$:

$$T(N, k) = T_{\text{init}} + T_{\text{loop}} = c_1(k \log k) + c_5(N \log k) \implies O(N \log k)$$

Summary of Big-O Time Complexities:

- **Worst-Case Time Complexity** — $O(N \log k)$: Occurs when elements are evenly distributed across all k streams, maintaining heap size near k throughout processing.
- **Best-Case Time Complexity** — $O(N \log k)$: Every pop and insert operation on the heap of size k costs $O(\log k)$ time regardless of input ordering.
- **Average-Case Time Complexity** — $O(N \log k)$: Optimal linear-logarithmic time relative to stream count k .

Space Complexity Derivation:

- **Auxiliary Heap Memory** — $O(k)$: The Min-Heap contains at most one element per active sub-array, requiring strictly $O(k)$ auxiliary space.
- **Output Vector Memory** — $O(N)$: Pre-reserving and storing all N elements in the `merged` vector requires $O(N)$ space.
- **Total Space Complexity**: $S(N, k) = O(k)$ auxiliary workspace, $O(N)$ total space including output memory.

8.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

9 Problem 9

Description: Ranking systems often need one order statistic without paying the cost of fully sorting a large dataset.

Question: Given an unsorted integer array and k , return the k^{th} -largest element. Duplicate values count as separate positions.

9.1 Problem Statement

Input: An unsorted integer array `arr` of size N , and an integer k ($1 \leq k \leq N$).

Task: Find the k^{th} -largest element in the array. Duplicate values are treated as distinct elements and occupy separate positional ranks.

Output: The integer value corresponding to the k^{th} -largest element.

Constraints: $N \geq 1$; $1 \leq k \leq N$; $arr[i] \in \mathbb{Z}$.

Edge Cases:

- $k = 1$: Returns the maximum element in the array.
- $k = N$: Returns the minimum element in the array.
- Array with duplicate elements (e.g., $[3, 2, 3, 1, 2, 4, 5, 5, 6]$ with $k = 4 \implies$ sorted as $[6, 5, 5, 4, 3, 3, 2, 2, 1]$, 4th-largest is 4).
- Array with negative integers (e.g., $[-1, -2, -3, -4]$ with $k = 2 \implies -2$).

9.2 Description of Approach

9.2.1 Algorithm Chosen and Correctness Explanation

While full array sorting solves the problem in $O(N \log N)$ time, a bounded Min-Heap achieves superior time performance without sorting the entire dataset:

1. **Bounded Min-Heap Construction** ($O(N \log k)$): We instantiate a min-priority queue (`priority_queue<int, vector<int>, greater<int>>`).
2. **Iterative Maintenance:** We iterate through each element in `arr`:
 - Push the current element onto the min-heap.
 - If the heap size exceeds k , pop the smallest element at the top.
3. **Result Extraction:** After processing all N elements, the min-heap retains exactly the k largest elements encountered across the entire array. Because it is a min-heap, the root element (`minheap.top()`) is the smallest among these top k values, which is by definition the k^{th} -largest element in the array.

Correctness Justification: Maintaining a min-heap capped at size k guarantees that any element smaller than the current k^{th} largest element is systematically evicted. Thus, the heap continuously holds the k largest values seen so far, leaving the k^{th} -largest element at the top when iteration completes.

9.2.2 Pseudocode

Algorithm 12 Problem 9 — Find k^{th} Largest Element via Bounded Min-Heap

```

1: procedure KTHLARGEST( $arr, N, k$ )
2:    $minheap \leftarrow$  empty Min Priority Queue
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:      $minheap.push(arr[i])$ 
5:     if  $minheap.size() > k$  then
6:        $minheap.pop()$  ▷ Evicts the smallest element in the current top  $k + 1$ 
7:     end if
8:   end for
9:   return  $minheap.top()$  ▷ Top element is the  $k^{\text{th}}$  largest value
10: end procedure

```

9.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of elements in the array and k be the target rank.

1. **Loop Iterations:** The loop executes exactly N times.
2. **Push Pop Operations:**
 - Pushing an element into a heap of size at most $k + 1$ takes $c_1 \cdot \log_2 k$ operations.
 - When the heap size reaches $k + 1$, popping the top element takes $c_2 \cdot \log_2 k$ operations.
3. **Top Extraction:** Accessing $minheap.top()$ takes $c_3 = O(1)$ constant time.

Total Exact Cost Function $T(N, k)$:

$$T(N, k) = N \cdot (c_1 \log_2 k + c_2 \log_2 k) + c_3 = (c_1 + c_2)N \log_2 k + c_3 \implies O(N \log k)$$

Summary of Big-O Time Complexities:

- **Worst-Case Time Complexity** — $O(N \log k)$: Occurs when elements continually cause sifting down in the heap of size k .
- **Best-Case Time Complexity** — $O(N \log k)$: Every insertion into the heap takes $O(\log k)$ regardless of input order.
- **Average-Case Time Complexity** — $O(N \log k)$: Significantly faster than full array sorting ($O(N \log N)$), especially when $k \ll N$.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(k)$: The min-heap contains at most $k + 1$ elements at any moment during execution, consuming strictly $O(k)$ extra space.
- **Total Space Complexity:** $S(N, k) = O(k)$ auxiliary workspace, $O(N)$ total space including input storage.

9.3 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

10 Problem 10

Description: An inversion measures how far an array is from sorted order and is used in ranking analysis and sequence comparison.

Question: Count pairs (i, j) for which $i < j$ and $\text{values}[i] > \text{values}[j]$. Develop an efficient solution by modifying Merge Sort or using an advanced data structure like a Fenwick Tree.

Example Input: [2, 4, 1, 3, 5] $\implies$ **Expected Output:** 3

Requirements: A brute-force $O(N^2)$ solution may be used for validation, but the submitted primary solution must run in $O(N \log N)$ time. Use a 64-bit integer for the inversion count.

10.1 Problem Statement

Input: An unsorted integer array `arr` of size N .

Task: Count all inversion pairs (i, j) such that $0 \leq i < j < N$ and $\text{arr}[i] > \text{arr}[j]$ using an efficient $O(N \log N)$ algorithmic strategy.

Output: The total count of inversions stored in a 64-bit integer (`long long`).

Constraints: $N \geq 0$; $\text{arr}[i] \in \mathbb{Z}$; count must prevent 32-bit integer overflow by using a 64-bit integer (`long long`).

Edge Cases:

- $N = 0$ or $N = 1$: Zero inversions (returns 0).
- Already sorted array (e.g., [1, 2, 3, 4, 5] $\implies$ 0 inversions).
- Reverse-sorted array (e.g., [5, 4, 3, 2, 1] $\implies \frac{N(N-1)}{2} = 10$ inversions).
- Array with duplicate elements (e.g., [2, 2, 2] $\implies$ 0 inversions, since $\text{arr}[i] > \text{arr}[j]$ is strict).
- Maximum inversion count exceeding 32-bit integer limits (e.g., $N = 10^5$ reverse-sorted yields $\sim 5 \times 10^9$ inversions, handled correctly by `long long`).

10.2 Approach 1: Modified Merge Sort (Divide and Conquer)

10.2.1 Algorithm Chosen and Correctness Explanation

While a nested double loop brute-force check yields $O(N^2)$ time, modifying ****Merge Sort**** computes inversions in optimal $O(N \log N)$ time using a divide-and-conquer strategy:

1. **Divide and Conquer Decomposition:** An inversion pair (i, j) in a split array can fall into one of three distinct categories:

- *Left Inversions:* Both indices i, j reside in the left sub-array $[\text{left} \dots \text{mid}]$.
- *Right Inversions:* Both indices i, j reside in the right sub-array $[\text{mid} + 1 \dots \text{right}]$.
- *Cross Inversions:* Index i lies in the left sub-array and j lies in the right sub-array such that $\text{arr}[i] > \text{arr}[j]$.

Total inversions equal: $\text{Total} = \text{Left_Inversions} + \text{Right_Inversions} + \text{Cross_Inversions}$.

2. **Efficient Cross-Inversion Counting during Merge Phase:** During the merge step, two sorted sub-arrays $L = \text{arr}[\text{left} \dots \text{mid}]$ and $R = \text{arr}[\text{mid} + 1 \dots \text{right}]$ are merged using pointers i and j :

- If $L[i] \leq R[j]$, no inversion exists for $L[i]$ relative to $R[j]$; $L[i]$ is copied to the temporary array `temp`.
- If $L[i] > R[j]$, then $R[j]$ is smaller than $L[i]$. Because sub-array L is already sorted, $R[j]$ is strictly smaller than **all remaining elements** from $L[i]$ up to $L[\text{mid}]$.
- Thus, exactly $(\text{mid} - i + 1)$ cross inversions are discovered simultaneously in $O(1)$ constant time.

Correctness Justification: Because sub-arrays are recursively sorted before merging, every pair of elements (i, j) is compared across a merge boundary exactly once. Counting $(\text{mid} - i + 1)$ when $\text{arr}[i] > \text{arr}[j]$ captures all cross-boundary inversions exhaustively without double counting.

10.2.2 Pseudocode

Algorithm 13 Problem 10 (Approach 1) — Count Inversions via Modified Merge Sort

```

1: function MERGEANDCOUNT(arr, temp, left, mid, right)
2:   i ← left,   j ← mid + 1,   k ← left
3:   count ← 0 ▷ 64-bit integer
4:   while i ≤ mid and j ≤ right do
5:     if arr[i] ≤ arr[j] then
6:       temp[k ← k + 1] ← arr[i ← i + 1]
7:     else
8:       temp[k ← k + 1] ← arr[j ← j + 1]
9:       count ← count + (mid − i + 1) ▷ Adds all remaining left-half elements
10:    end if
11:  end while
12:  while i ≤ mid do
13:    temp[k ← k + 1] ← arr[i ← i + 1]
14:  end while
15:  while j ≤ right do
16:    temp[k ← k + 1] ← arr[j ← j + 1]
17:  end while
18:  for idx ← left to right do
19:    arr[idx] ← temp[idx]
20:  end for
21:  return count
22: end function
23:
24: function COUNTINVERSIONSMERGESORT(arr, temp, left, right)
25:   count ← 0
26:   if left < right then
27:     mid ← left + (right − left) / 2
28:     count ← count + COUNTINVERSIONSMERGESORT(arr, temp, left, mid)
29:     count ← count + COUNTINVERSIONSMERGESORT(arr, temp, mid + 1, right)
30:     count ← count + MERGEANDCOUNT(arr, temp, left, mid, right)
31:   end if
32:   return count
33: end function

```

10.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of elements in the input array.

1. **Recurrence Relation:** The algorithm divides the array into two equal halves of size $\frac{N}{2}$, recursively counts inversions in both halves, and performs a linear merge pass of size N :

$$T(N) = 2T\left(\frac{N}{2}\right) + C \cdot N$$

2. **Applying Master Theorem:** For $T(N) = aT(N/b) + f(N)$ with $a = 2, b = 2, f(N) = O(N)$:

- $N^{\log_b a} = N^{\log_2 2} = N^1 = O(N)$.
- Since $f(N) = \Theta(N^{\log_b a})$, Case 2 of the Master Theorem applies.

$$T(N) = \Theta(N \log N)$$

3. **Merging Operations:** At each level of the $\lceil \log_2 N \rceil$ recursion tree, linear merging across all sub-arrays takes $c_1 \cdot N$ steps.

Total Exact Cost Function $T(N)$:

$$T(N) = c_1 \cdot N \log_2 N + c_2 N \implies O(N \log N)$$

Summary of Big-O Time Complexities:

- **Worst-Case Time Complexity** — $O(N \log N)$: Occurs on reverse-sorted inputs where the maximum number of cross-inversions are triggered.
- **Best-Case Time Complexity** — $O(N \log N)$: Merge Sort divides and merges unconditionally even if the input is already fully sorted.
- **Average-Case Time Complexity** — $O(N \log N)$: Performs $O(N \log N)$ operations on arbitrary or random inputs.

Space Complexity Derivation:

- **Auxiliary Memory Buffer** — $O(N)$: A single temporary array `temp` of size N is allocated upfront and reused across all recursive calls to perform linear merges, consuming $O(N)$ auxiliary space.
- **Recursion Stack Space** — $O(\log N)$: The maximum depth of the call stack for binary recursive splitting is $\lceil \log_2 N \rceil$, requiring $O(\log N)$ auxiliary space.
- **Total Space Complexity:** $S(N) = S_{\text{temp}} + S_{\text{stack}} = O(N) + O(\log N) \implies O(N)$ auxiliary workspace, $O(N)$ total space including input storage.

10.3 Approach 2: Fenwick Tree / Binary Indexed Tree (Range Query Approach)

10.3.1 Algorithm Chosen and Correctness Explanation

Instead of recursive divide-and-conquer, a **Fenwick Tree (Binary Indexed Tree)** transforms the inversion counting problem into a dynamic **Point Update and Range Prefix Query** sequence over a frequency array:

1. **Coordinate Compression** ($O(N \log N)$): Because array elements can be negative, non-contiguous, or outside standard indexing bounds, the array is copied, sorted, and deduplicated using `std::unique`. Each original value `arr[i]` is mapped to its 1-based rank $r \in [1, U]$ using binary search (`std::lower_bound`), where $U \leq N$ represents the count of unique elements.
2. **Right-to-Left Traversal and Dynamic Prefix Sums** ($O(N \log N)$): The array is processed **backwards** from index $i = N - 1$ down to 0:
 - For an element `arr[i]` mapped to rank r , every element currently inserted in the Fenwick Tree originated from an index $j > i$.
 - Executing a prefix query `query(r - 1)` calculates the sum of frequencies of elements with rank strictly less than r (i.e., elements to the right of i that are strictly smaller than `arr[i]`).
 - This frequency count is accumulated directly into the 64-bit variable `totalInversions`.
 - The current element's rank r is then inserted into the Fenwick Tree with a frequency increment of +1 via `update(r, 1)`.

Correctness Justification: Traversing from right to left guarantees that the Fenwick Tree only contains elements with index $j > i$. Querying range $[1, r - 1]$ isolates elements satisfying both constraints $i < j$ and `arr[i] > arr[j]`, yielding exact counts without double counting or missing pairs.

10.3.2 Pseudocode

Algorithm 14 Problem 10 (Approach 2) — Count Inversions via Fenwick Tree

```

1: procedure FENWICKUPDATE(BIT, size, index, delta)
2:   for  $i \leftarrow \text{index}; i \leq \text{size}; i \leftarrow i + (i \& (-i))$  do
3:      $\text{BIT}[i] \leftarrow \text{BIT}[i] + \text{delta}$ 
4:   end for
5: end procedure
6:
7: function FENWICKQUERY(BIT, index)
8:    $\text{sum} \leftarrow 0$ 
9:   for  $i \leftarrow \text{index}; i > 0; i \leftarrow i - (i \& (-i))$  do
10:     $\text{sum} \leftarrow \text{sum} + \text{BIT}[i]$ 
11:   end for
12:   return  $\text{sum}$ 
13: end function
14:
15: function COUNTINVERSIONSBIT(arr, N)
16:    $\text{sorted} \leftarrow \text{SortAndDeduplicate}(\text{arr})$ 
17:    $U \leftarrow \text{length}(\text{sorted})$ 
18:    $\text{BIT} \leftarrow \text{ArrayOfZeros}(U + 1)$ 
19:    $\text{inversions} \leftarrow 0$  ▷ 64-bit integer
20:   for  $i \leftarrow N - 1$  downto 0 do
21:      $\text{rank} \leftarrow \text{BinarySearch}(\text{sorted}, \text{arr}[i]) + 1$ 
22:      $\text{inversions} \leftarrow \text{inversions} + \text{FENWICKQUERY}(\text{BIT}, \text{rank} - 1)$ 
23:      $\text{FENWICKUPDATE}(\text{BIT}, U, \text{rank}, 1)$ 
24:   end for
25:   return  $\text{inversions}$ 
26: end function

```

10.3.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of elements in the input array and $U \leq N$ be the number of distinct elements.

1. **Coordinate Compression Step:** Sorting the auxiliary vector takes $O(N \log N)$ time, and deduplication via linear scanning takes $O(N)$ time.
2. **Main Loop Execution (N iterations):** For each element $\text{arr}[i]$:
 - Binary search rank lookup using `std::lower_bound` takes $O(\log U) \leq O(\log N)$.
 - Fenwick Tree prefix sum query `query(rank - 1)` visits at most $\lceil \log_2 U \rceil$ tree nodes, taking $O(\log N)$ time.
 - Fenwick Tree point update `update(rank, 1)` updates at most $\lceil \log_2 U \rceil$ tree nodes, taking $O(\log N)$ time.

Total cost across N loop iterations is $N \cdot [O(\log N) + O(\log N) + O(\log N)] = O(N \log N)$.

Summary of Big-O Time Complexities:

- **Worst-Case Time Complexity** — $O(N \log N)$: Governed by coordinate compression and N logarithmic updates/queries on reverse-sorted or arbitrary inputs.
- **Best-Case Time Complexity** — $O(N \log N)$: The right-to-left loop and tree updates execute unconditionally for all input configurations.

- **Average-Case Time Complexity** — $O(N \log N)$: Performs $O(N \log N)$ operations regardless of initial element permutations.

Space Complexity Derivation:

- **Coordinate Compression Buffer** — $O(N)$: Storing unique sorted elements requires an auxiliary array of size $U \leq N$.
- **Fenwick Tree Storage** — $O(N)$: The Fenwick Tree frequency array consumes size $U + 1 \leq N + 1$.
- **Total Space Complexity**: $S(N) = O(N) + O(N) = O(N)$ auxiliary workspace.

10.4 Test Cases and Screenshots

Please refer to the `results_valid.txt` file in the github link

10.5 Comparative Analysis: Merge Sort vs. Fenwick Tree

Both algorithms achieve identical asymptotic time complexity of $O(N \log N)$ and auxiliary space complexity of $O(N)$, but exhibit distinct practical trade-offs:

Metric / Feature	Modified Merge Sort	Fenwick Tree (BIT)
Time Complexity	$O(N \log N)$	$O(N \log N)$
Auxiliary Space	$O(N)$ (temp array + stack)	$O(N)$ (Compression + Tree array)
Pre-processing	None required	$O(N \log N)$ Coordinate Compression
Implementation Complexity	Low (standard divide-and-conquer)	Moderate (custom BIT + binary search)
Array Modification	Sorts input array in-place	Preserves original array order
Constant Factors	Lower (cache-friendly merges)	Higher (multiple vector passes)

Table 1: Comparison of Inversion Counting Strategies

Conclusion & Recommendation: For standard static array evaluation, **Modified Merge Sort is the superior choice** because it directly satisfies the assignment requirements, incurs smaller hidden constant factors due to cache-friendly sequential memory accesses during merges, and eliminates the extra memory passes required for coordinate compression. However, the Fenwick Tree approach offers greater flexibility for non-destructive querying or streaming/dynamic dataset extensions.